CSA1706-ARTIFICIAL INTELLIGENCE

G.AMRUTHA(192511250)

Write the python program to solve 8-Puzzle problem

```
from heapq import heappush, heappop

# Goal State
goal = ((1, 2, 3),
        (4, 5, 6),
        (7, 8, 0))

# Find blank tile (0)
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

# Manhattan Distance Heuristic
def heuristic(state):
    distance = 0
    for i in range(3):
        for j in range(3):
            value = state[i][j]
            if value != 0:
                goal_row = (value - 1) // 3
```

```
            goal_col = (value - 1) % 3
            distance += abs(i - goal_row) + abs(j - goal_col)
    return distance

# Generate neighboring states
def get_neighbors(state):
    neighbors = []
    x, y = find_blank(state)

    moves = [(-1,0), (1,0), (0,-1), (0,1)]

    for dx, dy in moves:
        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [list(row) for row in state]
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
            neighbors.append(tuple(tuple(row) for row in new_state))

    return neighbors

# A* Search Algorithm
def solve_puzzle(start):
    pq = []
    heappush(pq, (heuristic(start), 0, start, []))
    visited = set()

    while pq:
        f, g, state, path = heappop(pq)
```

```
        if state in visited:
            continue

        visited.add(state)
        path = path + [state]

        if state == goal:
            return path

        for neighbor in get_neighbors(state):
            if neighbor not in visited:
                h = heuristic(neighbor)
                heappush(pq, (g + 1 + h, g + 1, neighbor, path))

    return None

# Print solution
def print_solution(path):
    if path is None:
        print("No solution found.")
        return

    print("\nSolution found in", len(path)-1, "moves:\n")

    for step, state in enumerate(path):
        print("Step", step)
        for row in state:
            print(*row)
        print()
```

```
# ---------- Main Program ----------

print("Enter the Initial State (Use 0 for Blank Tile)")

initial = []

for i in range(3):
    row = list(map(int, input(f"Enter row {i+1}: ").split()))
    initial.append(tuple(row))

start = tuple(initial)

solution = solve_puzzle(start)

print_solution(solution)
```

INPUT:

Enter the Initial State (Use 0 for Blank Tile)

Enter row 1: 1 2 3

Enter row 2: 4 0 6

Enter row 3: 7 5 8

OUTPUT:

Solution found in 2 moves:

Step 0

1 2 3

4 0 6

7 5 8

Step 1

1 2 3

4 5 6

7 0 8

Step 2

1 2 3

4 5 6

7 8 0